

语法分析
自顶向下-递归下降

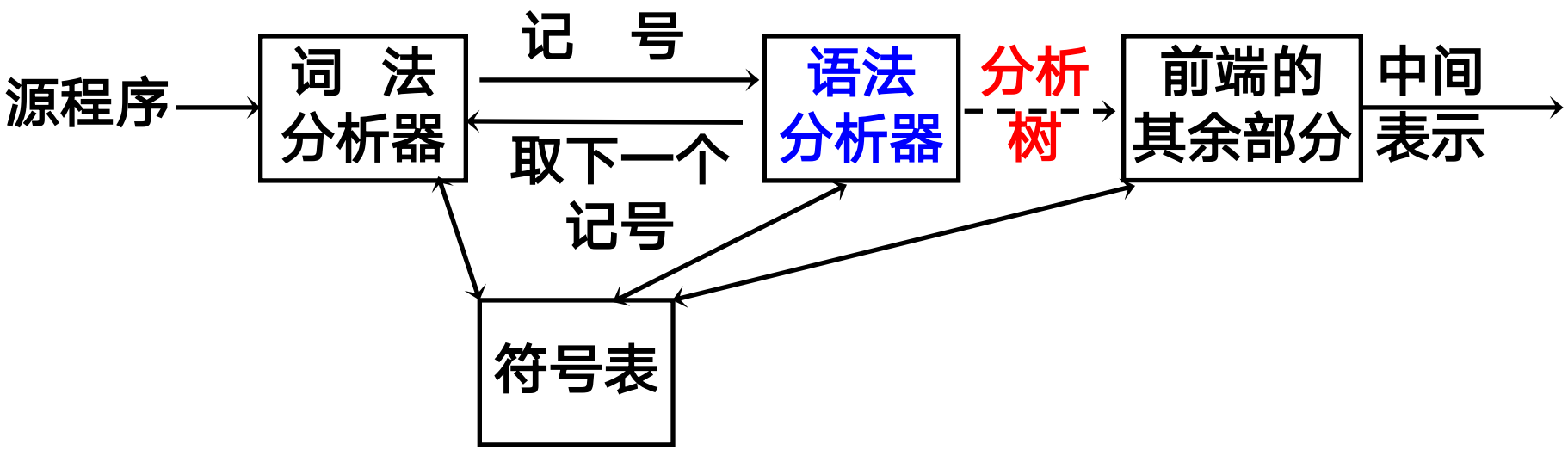
徐 伟

国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

计算机科学与技术学院

2026年09月24日

本节提纲



- 自顶向下与自底向上方法的区别
- 自顶向下分析方法
  - 递归下降分析方法
  - 消除左递归、提取左公因子

语法分析的主要方法

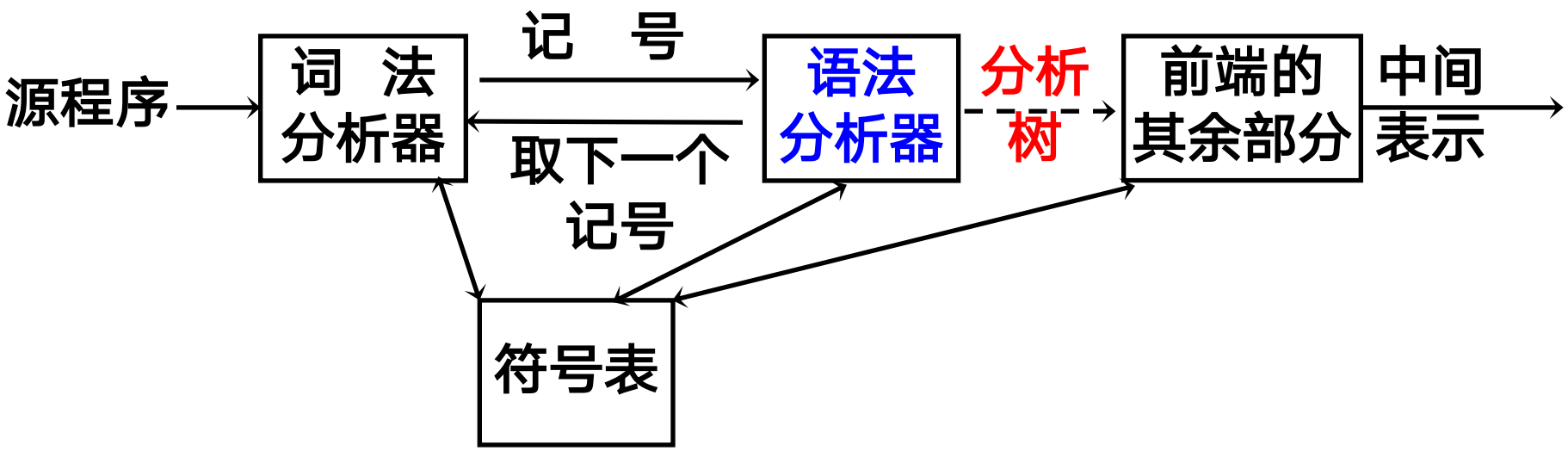
- **自顶向下（Top-down）**
  - 针对输入串，从文法的开始符号出发，尝试根据产生式规则推导（derive）出该输入串。

- **自底向上（Bottom-up）**
  - 针对输入串，尝试根据产生式规则归约（reduce）到文法的开始符号。

语法分析的主要方法

- **自顶向下（Top-down）**
  - 针对输入串，从文法的开始符号出发，尝试根据产生式规则推导（derive）出该输入串。
  - 分析树的构造方法
    - 从根部开始
- **自底向上（Bottom-up）**
  - 针对输入串，尝试根据产生式规则归约（reduce）到文法的开始符号。
  - 分析树的构造方法：
    - 从叶子开始

本节提纲



- 自顶向下与自底向上方法的区别
- **自顶向下分析方法**
  - 递归下降分析方法
  - 消除左递归、提取左公因子

递归下降语法分析

- **数据结构**
  - 一个输入缓冲区和向前看指针*lookahead*
- **分析过程**
  - 自左向右扫描输入串
  - 设计一个辅助过程*match()*，将*lookahead*指向的位置与产生式迭代生成的终结符进行匹配，如匹配，将*lookahead*挪到下一个位置
  - 为每一个非终结符写一个分析过程
    - 该过程可以调用其他非终结符的过程及match
    - 这些过程可能是递归的

递归下降语法分析——程序模拟推导

- 考虑以下文法：

$$expr \rightarrow term$$
$$\mid term + expr$$
$$\mid term - expr$$
$$term \rightarrow num \mid (expr)$$

- 分析过程：
  - 从左到右扫描输入串
  - 开始符号：*expr*
  - 按顺序尝试产生式

```
void expr() {
    term();
    if (lookahead == '+/-') {
        match('+/-');
        expr();
    }
    report("语法正确");
}

void term(){
    if (lookahead is num){
        match(lookahead);
    } else{ if (lookahead == '(') {
        match('(');
        expr();
        match(')');
    } else report("语法错误");}
}
```

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

expr → *term*

| *term* + *expr*

| *term* - *expr*

term → num | (*expr*)

(	2	−	3	)



expr

```
void expr() {
    term();
    if (lookahead == '+/−') {
        match('+/−');
        expr();
    }
    report("语法正确");
}

void term(){
    if (lookahead is num){
        match(lookahead);
    } else{ if (lookahead == '(') {
        match('(');
        expr();
        match(')');
    } else report("语法错误");}
}
```

递归下降语法分析——演示过程

- 考虑以下文法：

expr → *term*

| *term* + *expr*

| *term* - *expr*

term → num | (*expr*)

(	2	−	3	)



expr

|

term

```
void expr() {
    term();
    if (lookahead == '+/−') {
        match('+/−');
        expr();
    }
    report("语法正确");
}

void term(){
    if (lookahead is num){
        match(lookahead);
    } else{ if (lookahead == '(') {
        match('(');
        expr();
        match(')');
    } else report("语法错误");}
}
```



递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

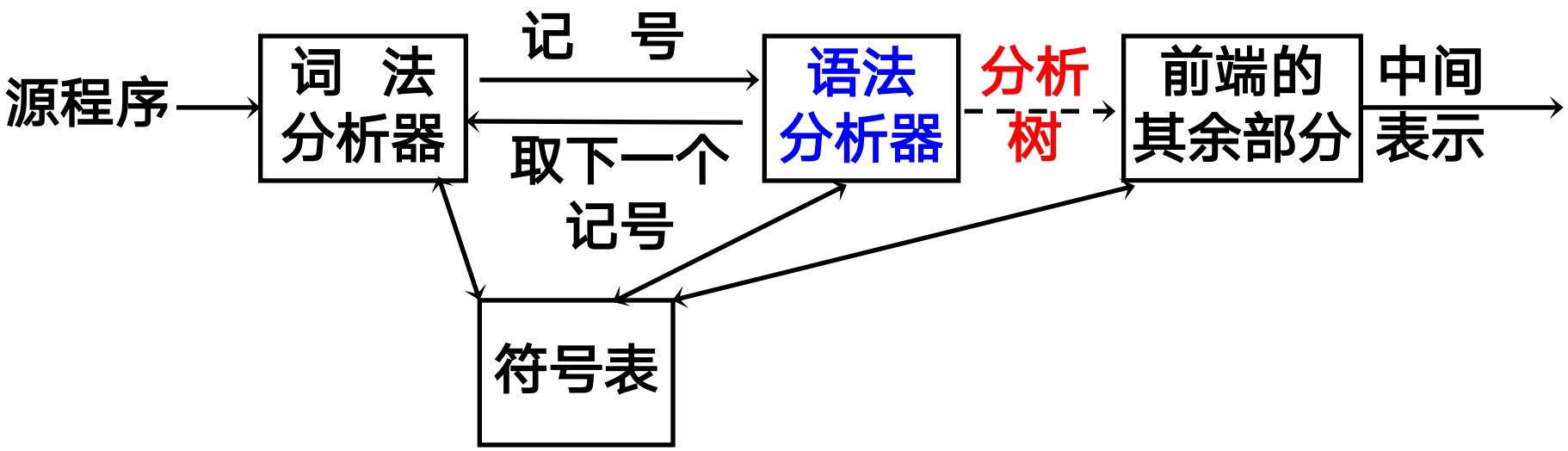
递归下降语法分析——演示过程

- 考虑以下文法：

$expr \rightarrow term$

$\quad\quad\quad

本节提纲



- 自顶向下与自底向上方法的区别
- **自顶向下分析方法**
  - 递归下降预测分析方法
  - 消除左递归、提取左公因子

递归下降的问题1

- 可能进入无限循环
- 考虑以下文法

$$S \rightarrow Sa \mid b$$

- 该文法是左递归的(left-recursive)

递归下降的问题1

- 可能进入无限循环
- 考虑以下文法

$$S \rightarrow Sa \mid b$$

- 该文法是左递归的(left-recursive)
- 自顶向下分析方法无法处理左递归
  - Why?

递归下降的问题1

- 可能进入无限循环
- 考虑以下文法

$$S \rightarrow Sa \mid b$$

- 该文法是左递归的(left-recursive)
- 自顶向下分析方法无法处理左递归
  - 考虑输入文法符号串为baaaaa
  - 最左推导如下：
    - $S \Rightarrow Sa \Rightarrow Saa \Rightarrow Saaa \Rightarrow Saaaa \ldots.$
    - 输入缓冲区lookahead指针纹丝未动

消除左递归

- **直接左递归**

  $A \to A\alpha \mid \beta$，**其中**α，β**不以A开头**

  - 串的特点 $\beta\alpha\ldots\alpha$ $(A \Rightarrow^{+} A\alpha)$
- **消除直接左递归**

  $$A \to \beta A'$$

  $$A' \to \alpha A' \mid \varepsilon$$

消除左递归

- 直接左递归

  $A \rightarrow A\alpha \mid \beta$，其中$\alpha$，$\beta$不以A开头

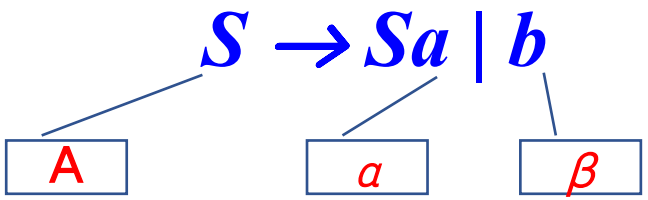
  ■ 串的特点 $\beta\alpha \ldots \alpha$ $(A \Rightarrow^{+} A\alpha)$

- 消除直接左递归

  $A \rightarrow \beta A'$

  $A' \rightarrow \alpha A' \mid \varepsilon$

- 考虑之前的文法



消除左递归

- 直接左递归

  $A \to A\alpha \mid \beta$，其中α，β不以A开头

  ■ 串的特点 $\beta\alpha \ldots \alpha$ $(A \Rightarrow^{+} A\alpha)$

- 消除直接左递归

  $A \to \beta A'$

  $A' \to \alpha A' \mid \varepsilon$

- 考虑之前的文法

$S \to Sa \mid b$

A　　α　　β

$S \to bS'$

$S' \to aS' \mid \varepsilon$

baaaaa推导：

$S \Rightarrow bS' \Rightarrow baS' \Rightarrow baaS' \Rightarrow baaaS' \Rightarrow baaaaS' \Rightarrow baaaaaS'$

输入缓冲区指针不停地移动

消除左递归

- 直接左递归

  $A \to A\alpha \mid \beta$，其中α，β不以A开头

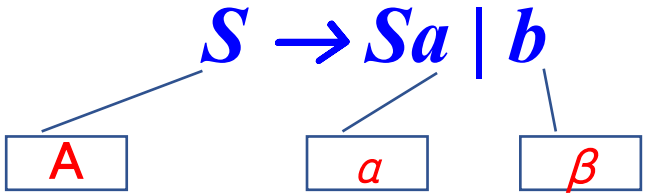
  ■ 串的特点 $\beta\alpha \ldots \alpha$ $(A \Rightarrow^{+} A\alpha)$

- 消除直接左递归

  $A \to \beta A'$

  $A' \to \alpha A' \mid \varepsilon$

- 考虑之前的文法



$S \to bS'$

$S' \to aS' \mid \varepsilon$

baaaaa推导：

$S \Rightarrow bS' \Rightarrow baS' \Rightarrow baaS' \Rightarrow baaaS' \Rightarrow baaaaS' \Rightarrow baaaaaS'$

输入缓冲区指针不停地移动

消除左递归

- 例 算术表达文法

$$E \to E + T \mid T$$

$$T \to T * F \mid F$$

$$F \to (E) \mid \mathbf{id}$$

消除左递归

- 例 算术表达文法

$$E \to E + T \mid T \qquad (\ T + T \ldots + T\ )$$

$$T \to T * F \mid F \qquad (\ F * F \ldots * F\ )$$

$$F \to (\ E\ ) \mid \mathrm{id}$$

消除左递归

- 例 算术表达文法

$$E \to E + T \mid T \qquad (\ T + T \ldots + T\ )$$
$$T \to T * F \mid F \qquad (\ F * F \ldots * F\ )$$
$$F \to (\ E\ ) \mid \mathrm{id}$$

$A \to A\alpha \mid \beta$
消除直接左递归
$A \to \beta\ A'$
$A' \to \alpha\ A' \mid \varepsilon$

- 消除左递归后文法

$$E \to TE'$$
$$E' \to + TE' \mid \varepsilon$$
$$T \to FT'$$
$$T' \to * F\ T' \mid \varepsilon$$
$$F \to (\ E\ ) \mid \mathrm{id}$$

注明：红色部分代表了α，蓝色部分代表了β

消除左递归的推广

- 处理任意数量的A产生式

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$$

其中β_i都不以A开头

改为：

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \ldots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \ldots \mid \alpha_m A' \mid \varepsilon$$

消除间接左递归

- 非直接左递归

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Sd \mid \varepsilon$$

消除间接左递归

- 非直接左递归

$$S \to Aa \mid b$$

$$A \to Sd \mid \varepsilon$$

- 先变换成直接左递归

$$S \to Aa \mid b$$

$$A \to Aad \mid bd \mid \varepsilon$$

- 再消除左递归

$$S \to Aa \mid b$$

$$A \to bdA' \mid A'$$

$$A' \to adA' \mid \varepsilon$$

$A \to A\alpha \mid \beta$

消除直接左递归

$A \to \beta A'$

$A' \to \alpha A' \mid \varepsilon$

递归下降的问题2

- 有左公因子的(left -factored)文法：
  - $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$

- 提左公因子(left factoring）
  - 推后选择产生式的时机，以便获取更多信息

$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ 等价于

$$A \rightarrow \alpha A'$$
$$A' \rightarrow \beta_1 \mid \beta_2$$

提左公因子(left factoring）

- 例 悬空*else*的文法

stmt → if *expr* then *stmt* else *stmt*

| if *expr* then *stmt*

| other

提左因子

stmt → if *expr* then *stmt optional_else_part*

| other

optional_else_part → else *stmt*

| ε

算法仍然二义！！！

递归下降的问题3

- **复杂的回溯→代价太高**
  - 非终结符有可能有多个产生式
  - 由于信息缺失，无法准确预测选择哪一个
  - 考虑到往往需要对多个非终结符进行推导展开，因此尝试的路径可能呈指数级爆炸
- **其分析过程类似于NFA**
- **问题：是否可以构造一个类似于DFA的分析方法？**

一起努力
打造国产基础软硬件体系！

徐 伟

国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

计算机科学与技术学院

2025年09月24日